```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/*
=====================================================
MENTAL SETUP (FIX THIS IN MIND)

Apple 🍎    = ERC20 Token A
Orange 🍊   = ERC20 Token B
ETH 💰      = Native coin
WETH 🪙     = ERC20 version of ETH (router handles)

Uniswap Rule:
x * y = k
=====================================================
*/

import "@uniswap/v2-core/contracts/interfaces/IUniswapV2Factory.sol";
import
"@uniswap/v2-periphery/contracts/interfaces/IUniswapV2Router02.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract UniswapV2CoreLearning {
    IUniswapV2Router02 public router;
    IUniswapV2Factory public factory;

    address public constant UNISWAP_ROUTER =
        0xeE567Fe1712Faf6149d80dA1E6934E354124CfE3;
    address public constant UNISWAP_FACTORY =
        0xF62c03E08ada871A0bEb309762E260a7a6a880E6;

    constructor() {
        router = IUniswapV2Router02(UNISWAP_ROUTER);
        factory = IUniswapV2Factory(UNISWAP_FACTORY);
    }

    function createPair(
        address tokenA, // Apple 🍎
        address tokenB // Orange 🍊
    ) external returns (address pair) {
```

```
        /*
    REAL LIFE:
    Tum kehte ho:
    "Mujhe Apple 🍎 aur Orange 🍊 ki
     ek nayi mandi (pool) kholni hai"

    Blockchain:
    - Factory check karta hai
    - Agar pehle se pair nahi hai
      to naya pair deploy hota hai
    */

        require(tokenA != tokenB, "Same token not allowed");
        require(tokenA != address(0) && tokenB != address(0), "Zero
address");

        pair = factory.createPair(tokenA, tokenB);

        /*
    Example:
    tokenA = Apple 🍎 address
    tokenB = Orange 🍊 address

    Output:
    pair = 0xABC...123 (new pool address)

    Ab isi address par:
    - liquidity add hogi
    - swaps honge
    */
    }

    function getPair(
        address tokenA, // Apple 🍎
        address tokenB // Orange 🍊
    ) external view returns (address pair) {
        /*
    REAL LIFE:
    Tum poochte ho:
    "Apple 🍎 aur Orange 🍊 ki mandi
```

```
     kahaan hai?"

    Agar mandi bani hogi → address milega
    Agar nahi bani → address(0)
    */

        pair = factory.getPair(tokenA, tokenB);

        /*
    Example:
    Apple 🍎 = 0xAAA
    Orange 🍊 = 0xBBB

    Output:
    pair = 0xABC...123

    Agar pair exist nahi karta:
    pair = 0x0000000000000000000000000000000000000000
    */
    }

    /*////////////////////////////////////////////////////////////////
    1️⃣ ADD LIQUIDITY (ERC20 + ERC20)
    ////////////////////////////////////////////////////////////////*/

    function addLiquidityERC20(
        address apple, // 🍎
        address orange, // 🍊
        uint appleAmount,
        uint orangeAmount,
        address to,
        uint deadline
    ) external returns (uint addedApple, uint addedOrange, uint liquidity)
{
        /*
        REAL EXAMPLE:
        User adds:
        100 Apple 🍎
        200 Orange 🍊
```

```
        Pool AFTER:
        Apple = 100
        Orange = 200

        Price:
        1 Apple = 2 Orange

        k = 100 * 200 = 20,000
        */

        IERC20(apple).transferFrom(msg.sender, address(this),
appleAmount);
        IERC20(orange).transferFrom(msg.sender, address(this),
orangeAmount);

        IERC20(apple).approve(address(router), appleAmount);
        IERC20(orange).approve(address(router), orangeAmount);

        /*
        🧠 TUMHARA SAMJHA HUA FLOW (✔️ CORRECT)

Tum keh rahe ho:

1️⃣ User ne pehle Apple aur Orange token ko
mere Uniswap helper contract ko approve kiya hoga

2️⃣ Isliye yahan transferFrom kaam kar raha hai

3️⃣ Contract tokens user se apne paas le leta hai

4️⃣ Uske baad contract Router ko approve deta hai

5️⃣ Router contract se tokens le kar pool me daal deta hai

👉 YES — 100% correct.

🔁 FULL TOKEN FLOW (STEP-BY-STEP)
🧑‍💻 STEP 0 — USER SIDE (VERY IMPORTANT)

User ne pehle se ye kiya hoga:
```

```
IERC20(apple).approve(uniswapHelper, 100);
IERC20(orange).approve(uniswapHelper, 200);


📌 Matlab:

"Main is helper contract ko allow karta hoon
mere Apple 🍎 aur Orange 🍊 lene ke liye"

🧱 STEP 1 — transferFrom (User → Helper Contract)
IERC20(apple).transferFrom(msg.sender, address(this), appleAmount);
IERC20(orange).transferFrom(msg.sender, address(this), orangeAmount);


Yahan hota kya hai?

User wallet
   ↓
Helper Contract


👉 Tokens ab user ke paas nahi,
👉 Contract ke paas aa gaye.

💡 Ye sirf isliye possible hai kyunki user ne approve kiya tha.

🧱 STEP 2 — approve (Helper Contract → Router)
IERC20(apple).approve(address(router), appleAmount);
IERC20(orange).approve(address(router), orangeAmount);


Yahan contract bol raha hai:

"Router bhai,
tum mere paas se
itne Apple 🍎 aur Orange 🍊 le sakte ho"


📌 Ye alag approve hai, user ka approve nahi.
```

🧱 STEP 3 — addLiquidity (Router → Pool)
router.addLiquidity(...)

Router internally:

Helper Contract → Pair (Pool)

Router:

tokens uthata hai

pool ke reserve update karta hai

LP tokens mint karta hai

🧱 STEP 4 — LP TOKENS
LP tokens → `to` address

📌 Usually:

to = msg.sender

🔥 VISUAL FLOW (SUPER IMPORTANT)

```
USER WALLET
   | (approve)
   ▼
HELPER CONTRACT
   | (approve)
   ▼
ROUTER
   ▼
PAIR (POOL)
```

Tumne exactly yahi describe kiya hai 👏

```
⚠️ IMPORTANT NUANCE (VERY IMPORTANT FOR GOOD DEV)
❓ Kya ye helper contract mandatory hai?

❌ Nahi

User direct router ko approve karke
direct router.addLiquidity bhi call kar sakta hai.

Tumhara helper contract ka fayda:

cleaner abstraction

reusable logic

frontend simple ho jaata hai
         */

        (addedApple, addedOrange, liquidity) = router.addLiquidity(
            apple,
            orange,
            appleAmount,
            orangeAmount,
            0,
            0,
            to,
            deadline
        );
    }

    /*////////////////////////////////////////////////////////////
    2️⃣ REMOVE LIQUIDITY (ERC20 + ERC20)
    ////////////////////////////////////////////////////////////*/

    function removeLiquidityERC20(
        address apple,
        address orange,
        uint liquidity,
        address to,
        uint deadline
    ) external returns (uint appleOut, uint orangeOut) {
```

```
        /*
        If user owns 100% LP:

        Pool:
        Apple = 100
        Orange = 200

        User gets back:
        100 Apple 🍎
        200 Orange 🍊
        */

        address pair = factory.getPair(apple, orange);

        IERC20(pair).transferFrom(msg.sender, address(this), liquidity);
        IERC20(pair).approve(address(router), liquidity);

        (appleOut, orangeOut) = router.removeLiquidity(
            apple,
            orange,
            liquidity,
            0,
            0,
            to,
            deadline
        );
    }
    /*
    🧑‍💻 STEP 0 — USER SIDE (VERY IMPORTANT)

User ke paas LP tokens hote hain
(ye LP tokens = pair contract ka ERC20 token)

User pehle se ye karta hai:

IERC20(pair).approve(uniswapHelper, liquidity);


📌 Matlab:
```

"Main is helper contract ko
apne LP tokens lene ka right deta hoon"

✔️ Tum yahan bilkul sahi ho.

🧱 STEP 1 — LP TOKENS: User → Helper Contract

```
IERC20(pair).transferFrom(msg.sender, address(this), liquidity);
```

Yahan kya hua?

```
User wallet
   ↓ (LP tokens)
Helper Contract
```

👉 LP tokens ab helper contract ke paas hain
👉 User ke paas LP tokens nahi rahe

✔️ Tumhara logic correct.

🧱 STEP 2 — LP TOKENS: Helper → Router (Approval)

```
IERC20(pair).approve(address(router), liquidity);
```

Yahan helper contract bol raha hai:

"Router bhai,
tum mere paas se itne LP tokens le sakte ho"

📌 Ye user ka approve nahi,
📌 Ye helper ka approve hai.

✔️ Tum yeh bhi sahi samajh rahe ho.

🧱 STEP 3 — Router burns LP tokens (IMPORTANT DETAIL)

```
router.removeLiquidity(...)
```

Router internally kya karta hai:

1️⃣ LP tokens leta hai
2️⃣ LP tokens burn karta hai ❌🔥
3️⃣ Pool ke reserves se tokens nikalta hai

⚠️ LP tokens transfer nahi hote,
⚠️ LP tokens burn hote hain

Ye ek important conceptual point hai.

🧱 STEP 4 — Tokens wapas milte hain

Router pool se nikalta hai:

Apple 🍎
Orange 🍊

Aur bhej deta hai:

to

Usually:

to = msg.sender

🔥 AB TUMHARI STATEMENT KO FIX KARTE HAIN
Tumne kaha 👇

"router mujhe mere dono token profit ke sath wapas deta hai"

Thoda correction 👇

✔️ Router tokens wapas deta hai
❌ Profit guarantee nahi hota

```
Kya milta hai?
Token amount = (LP share %) × (current pool reserves)


Kabhi zyada ho sakta hai

Kabhi kam ho sakta hai

👉 Ye depend karta hai:

swaps par

price movement par

impermanent loss par

🧠 REAL LIFE ANALOGY (FINAL LOCK 🔒)
LP token = mandi ki receipt 🧾

Receipt wapas do
↓
Receipt jala di jaati hai 🔥
↓
Mandi se jo fruit bacha hai
uska tumhara share milta hai

✅ FINAL FLOW (ONE SCREEN SUMMARY)
USER
│ approve LP
▼
HELPER CONTRACT
│ transferFrom LP
│ approve LP
▼
ROUTER
│ burn LP
▼
POOL
│ release Apple + Orange
▼
```

```
USER

     */

    /*////////////////////////////////////////////////////////////
    3️⃣ SWAP EXACT TOKENS (ERC20 → ERC20)
    ////////////////////////////////////////////////////////////*/

    function swapExactTokens(
        address apple,
        address orange,
        uint appleIn,
        uint orangeMin,
        address to,
        uint deadline
    ) external returns (uint orangeOut) {
        /*
        Pool BEFORE:
        Apple = 100
        Orange = 200
        k = 20,000

        User swaps:
        10 Apple 🍎

        New Apple = 110
        New Orange = 20,000 / 110 = 181.8

        Orange OUT:
        200 - 181.8 = 18.2 (≈ after fee)
        */

        IERC20(apple).transferFrom(msg.sender, address(this), appleIn);
        IERC20(apple).approve(address(router), appleIn);

        address;
        path[0] = apple;
        path[1] = orange;

        uint[] memory amounts = router.swapExactTokensForTokens(
```

```
            appleIn,
            orangeMin,
            path,
            to,
            deadline
        );

        orangeOut = amounts[1];
    }

    /*
    🔁 FUNCTION: swapExactTokens — DEEP WALKTHROUGH

Tumhara function (slightly formatted):

function swapExactTokens(
    address apple,
    address orange,
    uint appleIn,
    uint orangeMin,
    address to,
    uint deadline
) external returns (uint orangeOut)

🧠 PEHLE EK LINE ME MEANING

"Main exact Apple 🍎 de raha hoon,
mujhe kam se kam itne Orange 🍊 chahiye."

🧑‍💻 STEP 0 — USER SIDE (SAME PATTERN AS BEFORE)

User ne pehle se ye kiya hoga:

IERC20(apple).approve(uniswapHelper, appleIn);


📌 Matlab:

"Main is helper contract ko
itne Apple 🍎 lene ki permission deta hoon"
```

✔️ Same pattern as:

addLiquidity

removeLiquidity

🧱 STEP 1 — transferFrom (User → Helper Contract)

```
IERC20(apple).transferFrom(msg.sender, address(this), appleIn);
```

Yahan kya hua?

```
User wallet
   ↓ 10 Apple 🍎
Helper Contract
```

👉 Apple tokens ab helper contract ke paas hain
👉 User ke paas 10 Apple kam ho gaye

✔️ Ye possible sirf isliye hai kyunki user ne approve kiya tha.

🧱 STEP 2 — approve (Helper Contract → Router)

```
IERC20(apple).approve(address(router), appleIn);
```

Helper contract bol raha hai:

"Router bhai,
tum mere paas se
10 Apple 🍎 le sakte ho"

📌 Ye approve user ka nahi,
📌 Ye approve helper contract ka hai.

✔️ Same pattern as liquidity functions.

🧱 STEP 3 — path banana (VERY IMPORTANT)

❌ Tumhare code me chhota sa typo hai:

```
address;
path[0] = apple;
path[1] = orange;
```

✅ Correct version hona chahiye:

```
address;
path[0] = apple;
path[1] = orange;
```

Path ka matlab?
Apple 🍎 → Orange 🍊

👉 Router ko bataya:

"Kaunsa token de rahe ho
aur kaunsa chahiye"

🧱 STEP 4 — Router swap karta hai (MAIN ACTION)

```
uint[] memory amounts = router.swapExactTokensForTokens(
    appleIn,
    orangeMin,
    path,
    to,
    deadline
);
```

Router internally kya karta hai?
1️⃣ Helper se 10 Apple 🍎 leta hai
2️⃣ Pool ke reserves update karta hai
3️⃣ x * y = k follow karta hai
4️⃣ 0.3% fee cut karta hai
5️⃣ Orange 🍊 nikalta hai
6️⃣ Orange 🍊 `to` address ko bhejta hai

🔢 MATH (REAL VALUES — VERY IMPORTANT)

```
Pool BEFORE:
Apple = 100
Orange = 200
k = 20,000

User gives:
10 Apple 🍎

Pool AFTER:
Apple = 110
Orange = 20,000 / 110 = 181.8

Orange OUT:
200 - 181.8 = 18.2 Orange 🍊


➡️ Fee ke baad thoda kam (~18)

🔥 FULL TOKEN FLOW (DIAGRAM)
USER
| approve Apple
▼
HELPER CONTRACT
| transferFrom Apple
| approve Apple
▼
ROUTER
| swap via pool
▼
POOL (Apple ↔ Orange)
| x * y = k
▼
USER (gets Orange 🍊)
     */

    /*//////////////////////////////////////////////////////////////
    4️⃣ SWAP TOKENS FOR EXACT TOKENS
    //////////////////////////////////////////////////////////////*/

    function swapTokensForExact(
```

```
        address apple,
        address orange,
        uint orangeOut,
        uint appleMax,
        address to,
        uint deadline
    ) external returns (uint appleUsed) {
        /*
        User wants EXACT:
        20 Orange 🍊

        Pool:
        Apple = 100
        Orange = 200

        After removing 20 Orange:
        New Orange = 180

        New Apple = 20,000 / 180 = 111.1

        Apple IN:
        111.1 - 100 = ~11.1 Apple 🍎
        */

        IERC20(apple).transferFrom(msg.sender, address(this), appleMax);
        IERC20(apple).approve(address(router), appleMax);

        address;
        path[0] = apple;
        path[1] = orange;

        uint[] memory amounts = router.swapTokensForExactTokens(
            orangeOut,
            appleMax,
            path,
            to,
            deadline
        );

        appleUsed = amounts[0];
```

```
    }
    /*
    🔁 FUNCTION: swapTokensForExact — DEEP WALKTHROUGH

Tumhara function:

function swapTokensForExact(
    address apple,
    address orange,
    uint orangeOut,
    uint appleMax,
    address to,
    uint deadline
) external returns (uint appleUsed)

🧠 PEHLE EK LINE ME MEANING

"Mujhe EXACT itne Orange 🍊 chahiye,
main maximum itne Apple 🍎 dene ko ready hoon."

Ye swapExactTokens ka ulta version hai.

🧑‍💻 STEP 0 — USER SIDE (BILKUL SAME PATTERN)

User ne pehle se ye kiya hoga:

IERC20(apple).approve(uniswapHelper, appleMax);


📌 Matlab:

"Main is helper contract ko
maximum itne Apple 🍎 lene deta hoon"


✔️ Same as:

addLiquidity

removeLiquidity
```

swapExactTokens

🧱 STEP 1 — transferFrom (User → Helper Contract)

```
IERC20(apple).transferFrom(msg.sender, address(this), appleMax);
```

Yahan kya hua?

User wallet
↓ appleMax Apple 🍎
Helper Contract

👉 Helper contract ke paas ab maximum Apple aa gaye
👉 Actual me kitne use honge → baad me pata chalega

⚠️ Ye intentionally max amount liya jaata hai
taaki swap fail na ho.

🧱 STEP 2 — approve (Helper Contract → Router)

```
IERC20(apple).approve(address(router), appleMax);
```

Helper contract bol raha hai:

"Router bhai,
tum mere paas se
maximum itne Apple 🍎 le sakte ho"

✔️ Bilkul same approve logic as pehle.

🧱 STEP 3 — path banana (VERY IMPORTANT)

❌ Tumhare code me same typo hai jo pehle tha:

```
address;
path[0] = apple;
path[1] = orange;
```

```
✅ Correct conceptual form (samjhane ke liye):

address;
path[0] = apple;
path[1] = orange;

Path ka matlab?
Apple 🍎 → Orange 🍊


Router ko bata diya:

"Apple do, Orange chahiye"

🧱 STEP 4 — Router swap karta hai (MAIN DIFFERENCE)
uint[] memory amounts = router.swapTokensForExactTokens(
    orangeOut,
    appleMax,
    path,
    to,
    deadline
);

Yahan router ka behavior different hai:

Output fixed hai → orangeOut

Input variable hai → kitna Apple lagega

🔢 MATH (REAL VALUES — SAME AS COMMENT)
Pool BEFORE:
Apple = 100
Orange = 200
k = 20,000

User wants:
20 Orange 🍊 (EXACT)

Pool AFTER Orange removal:
New Orange = 200 - 20 = 180
```

```
Maintain k:
New Apple = 20,000 / 180 = 111.1

Apple IN:
111.1 - 100 = 11.1 Apple 🍎


➡️ Fee ke baad thoda zyada (~11.2)

🔥 FULL TOKEN FLOW (DIAGRAM)
USER
| approve Apple (max)
▼
HELPER CONTRACT
| transferFrom Apple (max)
| approve Apple (max)
▼
ROUTER
| calculates required Apple
| takes only needed Apple
| swaps via pool
▼
POOL (Apple ↔ Orange)
| x * y = k
▼
USER (gets EXACT Orange 🍊)

⚠️ IMPORTANT DIFFERENCE (VERY EXAM / INTERVIEW IMPORTANT)
swapExactTokens swapTokensForExact
Input fixed Output fixed
Output variable Input variable
"Main itna de raha" "Mujhe itna chahiye"
amounts[1] used amounts[0] used

     */

    /*//////////////////////////////////////////////////////////////
    5️⃣ SWAP EXACT ETH → TOKEN
    //////////////////////////////////////////////////////////////*/
```

```
    function swapExactETHForTokens(
        address apple,
        uint appleMin,
        address to,
        uint deadline
    ) external payable returns (uint appleOut) {
        /*
        Pool:
        ETH = 10
        Apple = 1000
        k = 10,000

        User sends:
        1 ETH

        New ETH = 11
        New Apple = 10,000 / 11 = 909

        Apple OUT:
        1000 - 909 = 91 Apple 🍎
        */

        address;
        path[0] = router.WETH(); // ETH → WETH
        path[1] = apple;

        uint[] memory amounts = router.swapExactETHForTokens{value:
msg.value}(
            appleMin,
            path,
            to,
            deadline
        );

        appleOut = amounts[1];
    }
    /*
    🔁 FUNCTION: swapExactETHForTokens — DEEP WALKTHROUGH
```

Tumhara function:

```
function swapExactETHForTokens(
    address apple,
    uint appleMin,
    address to,
    uint deadline
) external payable returns (uint appleOut)
```

🧠 PEHLE EK LINE ME MEANING

"Main EXACT ETH 💰 de raha hoon,
mujhe kam se kam itne Apple 🍎 chahiye."

Ye ETH → Token version hai
aur swapExactTokens ka ETH variant.

🏦 STEP 0 — USER SIDE (SABSE IMPORTANT DIFFERENCE)

User koi approve nahi karta ❌

User simply call karta hai:

```
swapExactETHForTokens{ value: 1 ether }(...)
```

📌 Kyun?

ETH ≠ ERC20
ETH approve nahi hota

👉 Yahi sabse bada difference hai ERC20 swap se.

🧱 STEP 1 — ETH CONTRACT ME AATA HAI
external payable

Aur frontend se:

```
msg.value = 1 ETH


Ownership flow:

USER
 ↓ 1 ETH
HELPER CONTRACT


👉 Ab 1 ETH helper contract ke paas hai

🧱 STEP 2 — PATH BANANA (ETH → WETH → APPLE)

Tumhare code me same typo hai (jaise pehle):

❌ Tumne likha:

address;
path[0] = router.WETH();
path[1] = apple;


✅ Conceptually correct form (samjhane ke liye):

address;
path[0] = router.WETH(); // ETH internally → WETH
path[1] = apple;         // WETH → Apple 🍎


📌 Yahan ETH ka naam bhi nahi likha
kyunki:

Router ETH ko WETH bana deta hai

🧱 STEP 3 — Router ko ETH dena (MOST IMPORTANT LINE)
router.swapExactETHForTokens{ value: msg.value }(...)


Yahan ETH direct router ko bheja ja raha hai.
```

```
Ownership flow:

HELPER CONTRACT
 ↓ 1 ETH
ROUTER

🧱 STEP 4 — ROUTER KA MAGIC (ETH → WETH)

Router internally ye karta hai:

1 ETH
 ↓ wrap
1 WETH


👉 Ye automatic hota hai
👉 Tum manually WETH ko touch hi nahi karte

🧱 STEP 5 — ACTUAL SWAP (POOL KE ANDAR)

Ab pool ke liye sab ERC20 hai:

WETH ↔ Apple 🍎

🔢 MATH (REAL VALUES — EXACT TUMHARE COMMENT JAISA)
Pool BEFORE:
ETH = 10   (actually WETH)
Apple = 1000
k = 10,000

User sends:
1 ETH

Pool AFTER:
New ETH = 11
New Apple = 10,000 / 11 = 909

Apple OUT:
1000 - 909 = 91 Apple 🍎
```

```
➡️ Fee ke baad thoda kam (~90)

🧱 STEP 6 — OUTPUT USER KO MILTA HAI

Router:

Apple 🍎 → to address


📌 Helper contract Apple ko hold nahi karta
📌 Direct user ko milta hai

🔥 FULL FLOW (VISUAL MENTAL DIAGRAM)
USER
│ sends ETH (msg.value)
▼
HELPER CONTRACT (payable)
│ forwards ETH
▼
ROUTER
│ wraps ETH → WETH
│ swaps via pool
▼
POOL (WETH ↔ Apple)
│ x * y = k
▼
USER (gets Apple 🍎)

⚠️ IMPORTANT DIFFERENCES (ERC20 SWAP vs ETH SWAP)
ERC20 swap  ETH swap
approve needed  ❌ no approve
transferFrom    ❌ no transferFrom
ERC20 only  payable + msg.value
path starts with token  path starts with WETH
🧠 appleMin KYUN HOTA HAI?
Slippage protection
```

```
Agar:

91 Apple expected
but sirf 85 mil rahe


➡️ swap revert ho jaata hai ❌
     */

    /*////////////////////////////////////////////////////////////
    6️⃣ SWAP EXACT TOKEN → ETH
    ////////////////////////////////////////////////////////////*/

    function swapExactTokensForETH(
        address apple,
        uint appleIn,
        uint ethMin,
        address to,
        uint deadline
    ) external returns (uint ethOut) {
        /*
        Pool:
        ETH = 11
        Apple = 909

        User gives:
        100 Apple 🍎

        New Apple = 1009
        New ETH = 10,000 / 1009 = 9.91

        ETH OUT:
        11 - 9.91 = 0.99 ETH
        */

        IERC20(apple).transferFrom(msg.sender, address(this), appleIn);
        IERC20(apple).approve(address(router), appleIn);

        address;
        path[0] = apple;
```

```
        path[1] = router.WETH();

        uint[] memory amounts = router.swapExactTokensForETH(
            appleIn,
            ethMin,
            path,
            to,
            deadline
        );

        ethOut = amounts[1];
    }
    /*
    🔁 FUNCTION: swapExactTokensForETH — DEEP WALKTHROUGH

Tumhara function:

function swapExactTokensForETH(
    address apple,
    uint appleIn,
    uint ethMin,
    address to,
    uint deadline
) external returns (uint ethOut)

🧠 PEHLE EK LINE ME MEANING

"Main EXACT Apple 🍎 de raha hoon,
mujhe kam se kam itna ETH 💰 chahiye."

Ye ERC20 → ETH version hai
aur swapExactETHForTokens ka ulta flow.

🧑‍💻 STEP 0 — USER SIDE (SAME OLD PATTERN)

User ne pehle se ye kiya hoga:

IERC20(apple).approve(uniswapHelper, appleIn);
```

📌 Matlab:

"Main is helper contract ko
itne Apple 🍎 lene ki permission deta hoon"

✔️ Same as:

addLiquidity

swapExactTokens

swapTokensForExact

🧱 STEP 1 — transferFrom (User → Helper Contract)

```
IERC20(apple).transferFrom(msg.sender, address(this), appleIn);
```

Ownership flow:

```
USER
 ↓ 100 Apple 🍎
HELPER CONTRACT
```

👉 Apple tokens ab helper contract ke paas
👉 User ke paas 100 Apple kam

✔️ Exactly same pattern as pehle.

🧱 STEP 2 — approve (Helper Contract → Router)

```
IERC20(apple).approve(address(router), appleIn);
```

Helper contract bol raha hai:

"Router bhai,
tum mere paas se
100 Apple 🍎 le sakte ho"

✔️ Ye approve user ka nahi,
✔️ Ye approve helper ka hai.

🧱 STEP 3 — path banana (TOKEN → WETH)

Tumhare code me same typo hai (jaise pehle):

❌ Tumne likha:

```
address;
path[0] = apple;
path[1] = router.WETH();
```

✅ Conceptually correct (samjhane ke liye):

```
address;
path[0] = apple;            // Apple 🍎
path[1] = router.WETH();    // ETH ka ERC20 form
```

📌 Matlab:

Apple 🍎 → WETH → ETH

⚠️ ETH directly pool me nahi jaata
pool sirf ERC20 (WETH) dekhta hai.

🧱 STEP 4 — ROUTER SWAP (MAIN ACTION)

```
uint[] memory amounts = router.swapExactTokensForETH(
    appleIn,
    ethMin,
    path,
    to,
    deadline
);
```

```
Router internally:

1⃣ Helper se Apple 🍎 leta hai
2⃣ Pool me Apple → WETH swap karta hai
3⃣ WETH ko unwrap karta hai → ETH
4⃣ ETH `to` address ko bhejta hai


👉 Tum WETH ko kabhi hold nahi karte
👉 Router sab handle karta hai

🔢 MATH (REAL VALUES — EXACT COMMENT KE JAISE)
Pool BEFORE:
ETH = 11
Apple = 909
k = 10,000

User gives:
100 Apple 🍎

Pool AFTER:
New Apple = 909 + 100 = 1009
New ETH   = 10,000 / 1009 = 9.91

ETH OUT:
11 - 9.91 = 0.99 ETH


➡️ Fee ke baad thoda kam (~0.98 ETH)

🔥 FULL TOKEN FLOW (MENTAL DIAGRAM)
USER
| approve Apple
▼
HELPER CONTRACT
| transferFrom Apple
| approve Apple
▼
ROUTER
| swap Apple → WETH
```

```
│ unwrap WETH → ETH
▼
USER (gets ETH 💰)

⚠️ IMPORTANT DIFFERENCES (ETH SWAP VS ERC20 SWAP)
ERC20 → ERC20   ERC20 → ETH
output ERC20    output ETH
no payable  ETH unwrap
tokens both sides   ETH only at end
pool sees ERC20 pool sees WETH
🧠 ethMin KYUN HOTA HAI?
Slippage protection


Agar:

Expected ≈ 0.99 ETH
But mil raha hai sirf 0.90


➡️ Transaction revert ho jaayega ❌
     */

    function swapETHForExactTokens(
        address apple, // 🍎 Token address
        uint appleOut, // EXACT Apple 🍎 chahiye
        address to, // jisko token mile
        uint deadline
    ) external payable returns (uint ethUsed) {
        /*
    🧠 USER INTENT:
    "Mujhe EXACT itne Apple 🍎 chahiye,
     main ETH 💰 jitna lage (max msg.value) dene ko ready hoon"

    ------------------------------------------------
    REAL POOL EXAMPLE:
    Pool:
    ETH = 10
    Apple = 1000
    k = 10,000
```

```
    User wants:
    100 Apple 🍎 (EXACT)

    Pool AFTER:
    Apple = 900
    ETH   = 10,000 / 900 = 11.11

    ETH USED:
    11.11 - 10 = 1.11 ETH

    Agar user ne msg.value = 2 ETH bheja:
    - 1.11 ETH use hua
    - ~0.89 ETH refund
    ------------------------------------------------
    */

        // -------------------------------
        // STEP 1️⃣ — PATH BANANA
        // ETH direct pool me nahi jaata
        // Router ETH → WETH banata hai
        // -------------------------------
        address;
        path[0] = router.WETH(); // ETH → WETH
        path[1] = apple; // WETH → Apple 🍎

        // -------------------------------
        // STEP 2️⃣ — ROUTER CALL
        // msg.value = max ETH willing to pay
        // -------------------------------
        uint[] memory amounts = router.swapETHForExactTokens{value:
msg.value}(
            appleOut, // EXACT output Apple 🍎
            path,
            to,
            deadline
        );

        /*
    amounts array ka matlab:
```

```
    amounts[0] = ETH actually used
    amounts[1] = Apple actually received (appleOut)
    */

        ethUsed = amounts[0];

        // NOTE:
        // Router automatically extra ETH refund kar deta hai
    }
    /*
🔁 1️⃣ swapETHForExactTokens

👉 ETH → Token (EXACT OUTPUT)

📌 FUNCTION
function swapETHForExactTokens(
    address apple,
    uint appleOut,   // EXACT Apple 🍎 chahiye
    address to,
    uint deadline
) external payable returns (uint ethUsed) {

🧠 ONE-LINE MEANING

"Mujhe EXACT itne Apple 🍎 chahiye,
main ETH 💰 maximum jitna lage dene ko ready hoon."

⚠️ Output fixed, input variable.

🧑‍💻 STEP 0 — USER SIDE

User frontend se call karega:

swapETHForExactTokens(
  apple,
  100 ether,
  user,
  deadline,
  { value: 2 ether }
);
```

📌 Matlab:

User bol raha:

“100 Apple chahiye, max 2 ETH de sakta hoon”

❌ No approve (ETH hai)
✅ payable + msg.value

🧱 STEP 1 — ETH CONTRACT ME AATA HAI
external payable

Ownership:

USER
↓ max ETH (msg.value)
HELPER CONTRACT

🧱 STEP 2 — PATH (ETH → WETH → APPLE)

```
address;
path[0] = router.WETH(); // ETH internally → WETH
path[1] = apple;
```

📌 Important:

Pool ETH nahi, WETH dekhta hai

Router wrap karega

🧱 STEP 3 — ROUTER CALL (CORE LOGIC)

```
uint[] memory amounts =
    router.swapETHForExactTokens{ value: msg.value }(
        appleOut,
        path,
        to,
        deadline
```

```
    );


Router internally:
1️⃣ ETH → WETH
2️⃣ Pool se exact Apple nikalta hai
3️⃣ Required ETH calculate karta hai
4️⃣ Extra ETH refund karta hai

🔢 REAL MATH (EXAMPLE)
Pool BEFORE
ETH = 10
Apple = 1000
k = 10,000

User wants
100 Apple 🍎 (EXACT)

Pool AFTER
New Apple = 900
New ETH   = 10,000 / 900 = 11.11

ETH USED
11.11 - 10 = 1.11 ETH


➡️ Agar user ne msg.value = 2 ETH bheja:

1.11 ETH use hua

~0.89 ETH refund

🧱 STEP 4 — amounts[] ARRAY
amounts[0] = ETH used (~1.11)
amounts[1] = Apple out (100)

ethUsed = amounts[0];

🔥 FINAL FLOW
USER
```

```
| sends ETH (max)
▼
HELPER (payable)
▼
ROUTER
| wrap ETH → WETH
| swap for exact Apple
| refund extra ETH
▼
USER (gets Apple 🍎)
 */

    function swapTokensForExactETH(
        address apple, // 🍎 Token address
        uint ethOut, // EXACT ETH 💰 chahiye
        uint appleMax, // max Apple 🍎 dene ko ready
        address to,
        uint deadline
    ) external returns (uint appleUsed) {
        /*
    🧠 USER INTENT:
    "Mujhe EXACT itna ETH 💰 chahiye,
     main max itne Apple 🍎 dene ko ready hoon"

    ------------------------------------------------
    REAL POOL EXAMPLE:
    Pool:
    ETH = 10
    Apple = 1000
    k = 10,000

    User wants:
    1 ETH (EXACT)

    Pool AFTER:
    ETH   = 9
    Apple = 10,000 / 9 = 1111.11

    Apple USED:
    1111.11 - 1000 = ~111 Apple 🍎
```

```
    ------------------------------------------------
    */

        // -------------------------------------
        // STEP 0⃣ — USER MUST HAVE DONE:
        // IERC20(apple).approve(this, appleMax)
        // -------------------------------------

        // -------------------------------------
        // STEP 1⃣ — USER → HELPER CONTRACT
        // -------------------------------------
        IERC20(apple).transferFrom(msg.sender, address(this), appleMax);

        // -------------------------------------
        // STEP 2⃣ — HELPER → ROUTER APPROVAL
        // -------------------------------------
        IERC20(apple).approve(address(router), appleMax);

        // -------------------------------------
        // STEP 3⃣ — PATH BANANA
        // Apple 🍎 → WETH → ETH
        // -------------------------------------
        address;
        path[0] = apple;
        path[1] = router.WETH();

        // -------------------------------------
        // STEP 4⃣ — ROUTER SWAP
        // -------------------------------------
        uint[] memory amounts = router.swapTokensForExactETH(
            ethOut, // EXACT ETH 💰
            appleMax, // max Apple 🍎 allowed
            path,
            to,
            deadline
        );

        /*
    amounts array:
    amounts[0] = Apple actually used
```

```
    amounts[1] = ETH received (ethOut)
    */

        appleUsed = amounts[0];

        // ⚠️ IMPORTANT:
        // Agar appleMax > appleUsed
        // leftover Apple contract me reh jaate hain
        // (production me refund karna chahiye)
    } /*
    🔁 2️⃣ swapTokensForExactETH

👉 Token → ETH (EXACT OUTPUT)

📌 FUNCTION
function swapTokensForExactETH(
    address apple,
    uint ethOut,     // EXACT ETH 💰 chahiye
    uint appleMax,   // max Apple 🍎 dene ko ready
    address to,
    uint deadline
) external returns (uint appleUsed) {

🧠 ONE-LINE MEANING

"Mujhe EXACT itna ETH 💰 chahiye,
main max itne Apple 🍎 dene ko ready hoon."

⚠️ ETH fixed, token variable.

🏠 STEP 0 — USER SIDE

User pehle approve karega:

IERC20(apple).approve(helper, appleMax);

🧱 STEP 1 — transferFrom (User → Helper)
IERC20(apple).transferFrom(
    msg.sender,
    address(this),
```

```
    appleMax
);


Ownership:

USER
 ↓ max Apple 🍎
HELPER CONTRACT

🧱 STEP 2 — approve (Helper → Router)
IERC20(apple).approve(address(router), appleMax);


📌 Same approve pattern as pehle sab swaps me.

🧱 STEP 3 — PATH (APPLE → WETH)
address;
path[0] = apple;
path[1] = router.WETH();


📌 Router end me WETH → ETH unwrap karega.

🧱 STEP 4 — ROUTER CALL
uint[] memory amounts =
    router.swapTokensForExactETH(
        ethOut,
        appleMax,
        path,
        to,
        deadline
    );


Router internally:
1️⃣ Pool se exact WETH nikalta hai
2️⃣ Required Apple calculate karta hai
3️⃣ WETH → ETH unwrap
4️⃣ ETH user ko bhejta hai
```

```
🔢 REAL MATH (EXAMPLE)
Pool BEFORE
ETH = 10
Apple = 1000
k = 10,000

User wants
1 ETH (EXACT)

Pool AFTER
New ETH   = 9
New Apple = 10,000 / 9 = 1111.11

Apple USED
1111.11 - 1000 = 111.11 Apple 🍎

🧱 STEP 5 — amounts[]
amounts[0] = Apple used (~111)
amounts[1] = ETH out (1 ETH)

appleUsed = amounts[0];

🔥 FINAL FLOW
USER
│ approve Apple
▼
HELPER
│ transferFrom Apple
│ approve Apple
▼
ROUTER
│ swap Apple → WETH
│ unwrap WETH → ETH
▼
USER (gets EXACT ETH 💰)

     */

    // Router unwrap ETH here
```

```
    receive() external payable {}
}
```